

P3279R0

CWG2463: What "trivially fooable" should mean

Published Proposal, 2024-05-15

Author:

[Arthur O'Dwyer](#)

Audience:

SG17

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

The core-language specification of "trivially copyable," "trivially copy constructible," and "trivially copy assignable" fail to match library-writers' expectations. In CWG2463, Core asked for a paper laying out the issues and proposing a direction. This is such a paper.

Table of Contents

1 Changelog

2 Three intuitions for TC

2.1 Library-writer's intuition

2.2 Compiler-writer's intuition

2.3 Abstract-machine-lawyer's intuition

2.4 Bottom line

3 Status quo

4 Surprising behaviors

- 4.1 TC types can have non-TC members
- 4.2 Types can act non-trivially while publicly advertising TC
- 4.3 "Trivial" Rule-of-Five functions can act unlike memcpy

5 Proposal

- 5.1 Conclusion

6 Acknowledgments

References

Informative References

§ 1. Changelog

- R0 (May 2024):
 - Initial revision. See relevant minutes from [Kona 2022](#), [Issaquah 2023](#).

§ 2. Three intuitions for TC

Different audiences think about trivial copyability (TC) in different ways. Here are three points of view I've identified: the library-writer, the compiler-writer, and the abstract-machine-lawyer.

§ 2.1. Library-writer's intuition

The library-writer's view is also shared by the trainer/instructor; it's the view that you need to have in order to understand most C++ codebases these days.

When a type is trivially copyable, all of its Rule-of-Five operations can be lowered to simple memcpy's. Furthermore, its Rule-of-Five operations are non-throwing and have no visible side effects.

This is the intuition by which every STL vendor writes things like this, which assumes that assignment of TC types can always be lowered to `memcpy`:

```
template<class T, class U>
U* std_copy(T* first, T* last, U* dfirst) {
    static_assert(is_assignable_v<T&, U&>);
    if constexpr (same_as<T, U> && is_trivially_copyable_v<T>) {
        memmove(dfirst, first, (last - first) * sizeof(T));
        return dfirst + (last - first);
    } else {
        while (first != last) *dfirst++ = *first++;
        return dfirst;
    }
}
```

And this, which assumes that assignment and construction can both be lowered:

```
template<class T>
void std_vector<T>::insert(std_vector<T>::const_iterator it, T value) {
    size_t i = it - begin();
    ensure_capacity();
    if constexpr (is_trivially_copyable_v<T>) {
        memmove(data() + i + 1, data() + i, size_ - i);
        size_ += 1;
    } else {
        ::new (data()[size_]) T(std::move(data()[size_ - 1]));
        size_ += 1;
        move(data() + i, data() + size_ - 1, data() + i + 1);
    }
    data()[i] = value;
}
```

A library might even assume that copying a TC type is "free" because it cannot have visible side effects:

```
template<class T, class Compare>
void sort3(T *x, T *y, T *z, Compare& c) {
    if constexpr (is_trivially_copyable_v<T> &&
                  is_constructible_v<T, T&> && is_assignable_v<T&, T&>) {
        // cond swap
        bool r = c(*z, *y);
```

```

T tmp = r ? *z : *y;
*z = r ? *y : *z;
*y = tmp;
// partially sorted swap
r = c(*z, *x);
tmp = r ? *z : *x;
*z = r ? *x : *z;
r = c(tmp, *y);
*x = r ? *x : *y;
*y = r ? *y : tmp;
} else {
    // use swap alone, which is slower
    if (c(*y, *x)) {
        if (c(*z, *y)) {
            swap(*x, *z);
        } else {
            swap(*x, *y);
            if (c(*z, *y)) {
                swap(*y, *z);
            }
        }
    }
    } else if (c(*z, *y)) {
        swap(*y, *z);
        if (c(*y, *x)) {
            swap(*x, *y);
        }
    }
}
}

```

The library-writer's intuition is the most fruitful, because it leads to all these cool optimizations. Unfortunately, we'll see that each of these optimizations has corner cases today which are unsafe, because the set of types actually considered TC by C++ includes some types for which these optimizations are unsafe.

§ 2.2. Compiler-writer's intuition

The compiler-writer's intuition is that all types start out "naturally" TC, and then get "made non-TC" in various ways. For example, if we see that any of `T`'s Rule-of-Five members are user-defined, we mark `T` as non-TC. If it has a subobject of non-TC type, we mark it non-TC. If it has a virtual base class, we mark it non-TC. And so on.

This paradigm is extremely easy to implement. It's also powerful and general: the compiler-writer can use the same technique to track "trivial relocatability," "trivial equality-comparability," "trivial value-initializability," etc. Unfortunately, we'll see that this paradigm doesn't work perfectly without some tweaks, because the set of types actually considered TC by C++ includes some types violating this paradigm. (For example, a TC type *can* have a non-TC type as a data member.)

§ 2.3. Abstract-machine-lawyer's intuition

Our previous two intuitions were driven by the physics of TC: what library operations can we do efficiently for TC types, and how can we efficiently compute the TC-ness of a type inside the compiler. The abstract-machine-lawyer's intuition is not driven by physics, but by the circular logic of the Standard itself. [\[basic.types.general\]/2–3](#) says:

2. For any [complete object] of TC type T [...] the underlying bytes making up the object can be copied into an array of char[, and when] that array is copied back into the object, the object shall subsequently hold its original value.
3. For two distinct [complete] objects obj1 and obj2 of TC type T, if the underlying bytes making up obj1 are copied into obj2, obj2 shall subsequently hold the same value as obj1.

This means that (by some mystical process) you can use `memcpy` to perform the same Platonic operation as assignment, even when `T` itself is not assignable. For example:

```
struct Stone {
    int i;
    Stone() {}
    Stone(Stone&&) = default;
    void operator=(const Stone&) = delete;
    void operator=(Stone&&) = delete;
    ~Stone() = default;
};

void via_assign(Stone& a, Stone& b) {
    a = b; // ill-formed
}
```

```
void via_memcpy(Stone& a, Stone& b) {
    std::memcpy(&b, &a, sizeof(Stone)); // OK, b now holds the same value as a
}
```

Over the years, CWG participants have suggested different metaphysical explanations for how `memcpy` does its job in such cases. For example, `memcpy` could "observe" that `Stone` is a TC type, and decide to perform the abstract-machine operations

```
b.~Stone(); // trivial
::new (&b) Stone(std::move(a)); // trivial
```

Moving-from `a` doesn't modify `a`'s value, because its move constructor is trivial.

The abstract-machine-lawyer doesn't care about whether that trivial move-constructor is selectable by overload resolution, whether it's ambiguous, or private, or anything else; it suffices for the abstract machine that we have sufficient non-deleted Rule-of-Five members for the abstract machine's `memcpy` to *somehow* take us from one state to the other.

[The minutes of CWG2463 discussion in Kona, November 2022](#) capture the abstract-machine-lawyer's position perfectly: "A class should be TC if there is a way [i.e., any way] to trivially copy or move it."

Consider again our `vector<T>::insert` from above, which in the library-writer's mind "uses `memmove` to create an object." The abstract-machine-lawyer knows that what "really" happens here is that `memmove` implicitly creates an object ([\[cstring.syn\]/3](#), [\[intro.object\]/11](#) so that when it copies the bytes into that object, the new object will take on the proper value ([\[basic.types.general\]/3](#)). The copying-of-bytes can still take place by destroying the implicitly created object and reconstructing a new object; the implicitly created object is "transparently replaceable" ([\[basic.life\]/8](#)) by the new object.

Now, if our TC type is not an implicit-lifetime type ([\[class.prop\]/9](#)), i.e. it is TC but has no eligible trivial constructors, then the above logic doesn't work; but in that case we *must* have an eligible trivial copy- or move-assignment operator (so `memcpy` can do its job by simply calling that assignment operator).

Unfortunately, as we'll see below, today it is possible to create a TC type that is copy-constructible without giving it an eligible copy constructor. Such a TC type can be put into a `vector`, and used with `vector::insert`, even though it is not an implicit-lifetime type. This will be a problem for the abstract-machine-lawyer's intuition.

§ 2.4. Bottom line

There is a tension between the library-writer's intuition and the abstract-machine-lawyer's intuition.

The library-writer wants C++ to report types as "TC" only if their Rule-of-Five operations can be safely lowered to `memcpy`. False positives are deadly. Basically, the "TC" label tells the library-writer that shuffling bytes instead of objects *preserves all library-relevant value-semantic behavior*.

The abstract-machine-lawyer wants C++ to report types as "non-TC" only if it would never make sense to `memcpy` them at all, because today it is UB to `memcpy` anything unless it's TC. Basically, the "TC" label is what permits legally shuffling bytes instead of objects *for any reason*.

The library-writer wants a strictly conservative "TC" label, with no false positives, because each false positive means misbehavior at runtime. (We'll see some false positives below.) The abstract-machine-lawyer tends to want a liberal "TC" label, with as few as possible false negatives, because each false negative means a program that works correctly in practice, but formally has UB.

§ 3. Status quo

The Standard defines several kinds of "triviality." Let's cover each of them quickly.

1. `is_trivial<T>` ([[class.prop](#)]/2, [[basic.types.general](#)]/9, [[meta.unary.prop](#)]). This is just TC plus an eligible trivial default constructor. It is not useful in practice (like `is_literal`) and we're getting rid of it via [P3247](#). It doesn't need to change.

2. `is_trivially_copyable<T>` ([[class.prop](#)]/1, [[basic.types.general](#)]/9, [[meta.unary.prop](#)]).

A *trivially copyable class* is a class:

- that has at least one eligible copy constructor, move constructor, copy assignment operator, or move assignment operator,
- where each eligible copy constructor, move constructor, copy assignment operator, and move assignment operator is trivial, and
- that has a trivial, non-deleted destructor.

Scalar types, trivially copyable class types, arrays of such types, and cv-qualified versions of these types are collectively called *trivially copyable types*.

3. `is_trivially_constructible<T, Args...>` ([[meta.unary.prop](#)], [[meta.unary.prop](#)]/9) and `is_trivially_assignable<T, U>` ([[meta.unary.prop](#)]).

[`is_trivially_constructible<T, Args...>` is true whenever] the variable definition `T t(declval<Args>()...);` is well-formed and is known to call no operation that is not trivial. Access checking is performed as if in a context unrelated to `T` and any of the `Args`.

[`is_trivially_assignable<T, U>` is true whenever] the expression `declval<T>() = declval<U>()` is well-formed when treated as an unevaluated operand. and is known to call no operation that is not trivial. Access checking is performed as if in a context unrelated to `T` and `U`.

Now, this wording depends on the definition of "trivial operation," which the Standard conspicuously fails to define. In practice, compiler vendors treat every operation as trivial *unless* it is a function call, in which case it is trivial if-and-only-if it calls a trivial special member function (as defined below).

Notably, `is_trivially_constructible<int, float>` and `is_trivially_constructible<bool, int>` are true (because they do not call functions) despite not doing anything like a `memcpy`. Aggregate initialization is also trivial:

```
struct Agg { int i; };
static_assert(std::is_trivially_constructible_v<Agg, int>);
static_assert(std::is_trivially_constructible_v<Agg, float>);
```

NOTE: There is implementation divergence on lambda conversions, which are arguably "known" to the compiler despite acting like user-defined conversion functions. EDG+MSVC say that `is_trivially_constructible<void(*)(), decltype([]{})>` is true; Clang+GCC say false. EDG+MSVC say that `is_trivially_assignable<void(*&)(), decltype([]{})>` is true; Clang+GCC say false.

NOTE: There is implementation divergence on NSDMIs, which are arguably "known" to the compiler. Given `struct X { int a=42; }` and `struct Q { X a, b; }`, EDG+GCC+MSVC say that `is_trivially_constructible<Q, X>` is true; Clang says that it's false. (EDG keeps it true even when you change 42 to any arbitrary computation `f()`; that's likely accidental.)

Given `struct E {}` and `struct R { E a; int b=42; }`, all vendors agree that `is_trivially_constructible<R, E>` is true. Yet `is_trivially_constructible<R>` is false, because it calls a non-trivial default constructor ([\[class.default.ctor\]/3](#)).

NOTE: There is likely-accidental implementation divergence on aggregate initialization from multiple arguments. Given `struct P { int a, b; }`, EDG says that `is_trivially_constructible<P, int, int>` is false; Clang+GCC+MSVC say true. We propose to deprecate the three-argument type trait, anyway.

5. A special member function is *trivial* in specific circumstances ([\[class.default.ctor\]/3](#), [\[class.copy.ctor\]/11](#), [\[class.copy.assign\]/9](#), [\[class.dtor\]/8](#)).

A default constructor is *trivial* if it is not user-provided and if:

- its class has no virtual functions and no virtual base classes, and
- no non-static data member of its class has a default member initializer, and
- all the direct base classes of its class have trivial default constructors, and
- for all the non-static data members of its class that are of class type (or array thereof), each such class has a trivial default constructor.

A copy/move constructor for class *X* is *trivial* if it is not user-provided and if:

- class *X* has no virtual functions and no virtual base classes, and
- the constructor selected to copy/move each direct base class subobject is trivial, and
- for each non-static data member of *X* that is of class type (or array thereof), the constructor selected to copy/move that member is trivial.

A copy/move assignment operator for class *X* is *trivial* if it is not user-provided and if:

- class *X* has no virtual functions and no virtual base classes, and
- the assignment operator selected to copy/move each direct base class subobject is trivial, and
- for each non-static data member of *X* that is of class type (or array thereof), the assignment operator selected to copy/move that member is trivial.

A destructor is *trivial* if it is not user-provided and if:

- the destructor is not virtual,
- all of the direct base classes of its class have trivial destructors, and
- for all of the non-static data members of its class that are of class type (or array thereof), each such class has a trivial destructor.

NOTE: [class.default.ctor] conspicuously doesn't require that "the constructor selected to initialize that member is trivial"; it simply requires that each base and member have a (possibly ineligible) trivial default constructor. Naturally, there is implementation divergence here ([Godbolt](#)). EDG seems to follow the letter of the law; Clang+GCC+MSVC hew closer to the library-writer's intuition.

Notice that `is_trivially_constructible`, `is_trivially_assignable`, [class.copy.ctor], and [class.copy.assign] check the results of overload resolution, i.e., they are friendly to the library-writer's intuition that what matters is not what *special member functions* happen to be defined, but which (possibly non-special) functions are actually called by overload resolution for a given C++ expression. Vice versa, `is_trivially_copyable` and [class.default.ctor] are hostile to the library-writer's intuition; they are defined only in terms of special member functions, ignoring overload resolution.

Two more critical things to keep in mind for the following examples:

- A copy constructor, copy assignment operator, etc, is never a template. So you can have a poorly-matching trivial copy constructor living alongside a non-trivial constructor template that is a better match; yet this will not cause `is_trivially_copyable` to become false, because a constructor template is not a copy constructor. All of your copy constructors remain trivial.
- The rule for calling a special member function "trivial" is context-independent. Once trivial, always trivial. So if you inherit a member function from your base class, it will continue to "be trivial" for you, just like it was trivial for your base class.

§ 4. Surprising behaviors

§ 4.1. TC types can have non-TC members

This is [\[CWG2463\]](#) ([Godbolt](#)):

```
struct Hamlet {
    Hamlet(const Hamlet&) { puts("copied"); }
    Hamlet(Hamlet&&) = default;
    Hamlet& operator=(Hamlet&&) = default;
};
struct Nutshell {
    Hamlet h_;
    Nutshell(Nutshell&&) = default;
    Nutshell& operator=(Nutshell&&) = default;
```

```
};
static_assert(!std::is_trivially_copyable_v<Hamlet>);
static_assert(std::is_trivially_copyable_v<Nutshell>);
```

NOTE: There is implementation divergence. EDG+MSVC say `std::is_trivially_copyable<Nutshell>` is true; Clang+GCC say false. EDG+MSVC are certainly correct according to the Standard.

To paraphrase Daveed Vandevoorde's comment on that issue: "It is unclear why a complete object of type `Hamlet` cannot be memcpied but such an object can be memcpied when embedded in a `Nutshell`." That is, this example is problematic to the abstract-machine-lawyer intuition that TC-ness ought to follow the *definedness* of `memcpy`. It doesn't really make sense to say that `memcpy` is capable of transferring the value of a `Hamlet` object only when it's a subobject of a `Nutshell` and not otherwise.

But to the library-writer intuition, this example is perfectly sane: TC means "all provided operations are tantamount to `memcpy`," which is clearly untrue of `Hamlet` and clearly true of `Nutshell` — because `Nutshell` provides fewer operations. `Nutshell` snips away `Hamlet`'s non-trivial copy constructor, so that all the remaining operations are in fact trivial. We need `Hamlet` to remain non-TC so that we don't misoptimize `std::uninitialized_copy(Hamlet)`; but we are happy for `Nutshell` to be TC so that we can optimize `std::uninitialized_move(Nutshell)`. We aren't worried about misoptimizing `std::uninitialized_copy(Nutshell)` because that's already ill-formed diagnostic required.

Finally, this example is problematic for the compiler-writer intuition. We want to be able to track a single bit, "`Hamlet` is non-TC," and propagate that down into `Nutshell`. But this example shows that we must track *exactly why* `Hamlet` is non-TC! This `Hamlet2` differs only in that it has a non-trivial move-assignment operator ([Godbolt](#)):

```
struct Hamlet2 {
    Hamlet2(const Hamlet2&) { puts("copied"); }
    Hamlet2(Hamlet2&&) = default;
    Hamlet2& operator=(Hamlet2&&); // not defaulted
};
struct Nutshell {
    Hamlet2 h_;
    Nutshell(Nutshell&&) = default;
    Nutshell& operator=(Nutshell&&) = default;
```

```
};
static_assert(!std::is_trivially_copyable_v<Hamlet2>);
static_assert(!std::is_trivially_copyable_v<Nutshell>);
```

Non-TC `Hamlet` embedded in a `Nutshell` becomes TC; non-TC `Hamlet2` embedded in the same `Nutshell` stubbornly remains non-TC. So it's not the TC-ness of `Hamlet` that matters; it's something else — something that takes more than one bit to represent in the compiler.

§ 4.2. Types can act non-trivially while publicly advertising TC

Microsoft STL's `std::pair<int&, int&>` advertises itself as TC, despite not modeling trivial copyability. To falsely advertise yourself as TC, recall that there are several possible signatures for a "copy assignment operator," and user-declaring any such signature (even an out-of-the-way one, even deleted) suffices to keep the compiler from implicitly generating any defaulted `operator=` for your class. Meanwhile, you provide assignment from `const Plum&` via a template (recall that the compiler never considers a template to be a Rule-of-Five special member). [Godbolt](#):

```
struct Plum {
    int *ptr_;
    explicit Plum(int& i) : ptr_(&i) {}
    Plum(const Plum&) = default;
    void operator=(const volatile Plum&) = delete;
    template<class=void> void operator=(const Plum& rhs) {
        *ptr_ = *rhs.ptr_;
    }
};
static_assert(std::is_trivially_copyable_v<Plum>);
static_assert(std::is_assignable_v<Plum&, const Plum&>);
static_assert(!std::is_trivially_assignable_v<Plum&, const Plum&>);
```

Microsoft STL's `std::reverse_copy` will see that `Plum` is trivially copyable and assume (following the library-writer's intuition) that it can be lowered to `memcpy`. So `std::reverse_copy(Plum)` is miscompiled. (By luck, `std::reverse_copy(pair)` is miscompiled only on 32-bit platforms, because Microsoft's `reverse_copy` optimization is also gated on `sizeof(T) <= 8`.) libstdc++ will miscompile `std::copy(Plum)` ([bug #114817](#)) and several other algorithms.

This example isn't problematic for the compiler-writer nor the abstract-machine-lawyer. But it is shocking to everyone's intuition because it shows that we can easily use corner cases in the wording to

decouple the "advertised" TC-ness of a type from its "proper" semantic TC-ness. (See [\[Müller\]](#) and [\[ODwyer\]](#).)

Show this example to the library-writer and when they recover from the shock they'll say: "Okay, so we shouldn't gate our `copy` and `reverse_copy` optimizations on TC. We'll gate them on the more granular `is_trivially_copy_assignable` trait, instead." The next section shows how that gate, also, is broken.

§ 4.3. "Trivial" Rule-of-Five functions can act unlike `memcpy`

The rule for calling a special member function "trivial" lacks context. Once trivial, always trivial. So if you inherit a member function from your base class, it will continue to "be trivial" for you, just like it was trivial for your base class. This allows us to make class `Leopard` which never changes its `spots_` — and yet, the type-traits report that everything about it is trivial ([Godbolt](#)):

```
struct Cat {};  
struct Leopard : Cat {  
    int spots_;  
    Leopard& operator=(Leopard&) = delete;  
    using Cat::operator=;  
};  
static_assert(is_trivially_copyable_v<Leopard>);  
static_assert(is_trivially_assignable_v<Leopard&, const Leopard&>);  
  
void test(Leopard& a, const Leopard& b) {  
    a = b; // absolutely no data is copied  
}
```

This is supremely problematic for the library-writer. Every STL vendor miscompiles `std::copy(Leopard)` —they assume that because `is_trivially_copy_assignable_v<Leopard>`, therefore copying a contiguous range of `Leopard` can be lowered to `memmove`. But in fact that's wrong.

I see no way to fix everybody's `std::copy(Leopard)` on the library side. The only way forward is to fix the definition of triviality in the core language.

§ 5. Proposal

Basically, we propose to respecify all of the traits to conform with the library-writer's intuition. (For each case below, read "can be" as "is known to the compiler to be able to be".)

- `is_trivially_constructible<T>` should mean precisely that constructing a `T` from no arguments can be lowered to a no-op. (This is basically the status quo in practice.)
- `is_trivially_constructible<To, From>` should mean precisely that constructing a (complete) `To` from a `From` can be lowered to memcopy. (E.g. `is_trivially_constructible<float, int>` should become false; but `is_trivially_copy_constructible<Polymorphic>` should become frequently true.)
- `is_trivially_constructible<To, Args...>` for any number of `Args...` greater than 1 should be invariably false, and we should deprecate it. That trait is useless because it corresponds to nothing physical.
- `is_trivially_destructible<T>` should mean precisely that destroying a `T` can be lowered to a no-op. (This is the status quo in practice.)
- `is_trivially_assignable<To, From>` should mean precisely that assigning a (complete) `To` from a `From` can be lowered to memcopy. (E.g. `is_trivially_assignable<float&, int&>` should become false.)
- `is_trivially_copyable<T>` should mean that all the Rule-of-Five operations that `T` supports, no matter the cvref-qualification of the arguments, can be lowered to memcopy. That is, it should mean exactly:

```
constexpr bool implies(bool a, bool b) { return b || !a; }
template<class T>
inline constexpr bool is_trivially_copyable_v =
    implies(is_constructible_v<T, T&>,      is_trivially_constructible_v<T, T&>) &&
    implies(is_constructible_v<T, T&&>,      is_trivially_constructible_v<T, T&&>) &&
    implies(is_constructible_v<T, const T&>,  is_trivially_constructible_v<T, const T&>) &&
    implies(is_constructible_v<T, const T&&>, is_trivially_constructible_v<T, const T&&>) &&
    implies(is_assignable_v<T&, T&>,         is_trivially_assignable_v<T&, T&>) &&
    implies(is_assignable_v<T&, T&&>,         is_trivially_assignable_v<T&, T&&>) &&
    implies(is_assignable_v<T&, const T&>,    is_trivially_assignable_v<T&, const T&>) &&
    implies(is_assignable_v<T&, const T&&>,    is_trivially_assignable_v<T&, const T&&>) &&
    implies(is_destructible_v<T>,            is_trivially_destructible_v<T>);
```

Each of the above definitions models the library-writer's intuition that "`T` is trivially fooable" means no more or less than "the foo operation on `T` can be lowered to the foo operation on `T`'s object representation." (With the special case that TC refers to *all* supported value-semantic operations, as a

package deal. Something like this definition of TC was independently suggested by Casey Carter in [an August 2020 PR comment](#).) This intuition applies uniformly to every "triviality" trait in today's Standard Library, and several more that have been proposed and/or implemented by vendors:

- trivially default-constructible (*sic*; read "default-initializable")
- trivially value-initializable ([P2782])
- trivially (copy|move)-constructible
- trivially (copy|move)-assignable
- trivially destructible
- trivially equality-comparable (Clang's `__is_trivially_equality_comparable` builtin)
- trivially lexicographically-comparable (defined internally by libc++)
- trivially swappable

The wording might look like:

```
[is_trivially_constructible<T, Args...> is true if and only if] is_constructible_v<T, Args...> is true and the variable definition for is_constructible, as defined below, is known to call no operation that is not trivial be equivalent in its observable effects to a simple copy of the complete object representation (when sizeof...(Args) == 1) or to have no observable effects (when sizeof...(Args) == 0). When sizeof...(Args) >= 2, yields false; this usage is deprecated.
```

```
[is_trivially_assignable<T, U> is true if and only if] is_assignable_v<T, U> is true and the assignment, as defined by is_assignable, is known to call no operation that is not trivial be equivalent in its observable effects to a simple copy of the complete object representation.
```

```
[is_trivially_destructible<T> is true if and only if] is_destructible_v<T> is true and remove_all_extents_t<T> is either a non-class type or a class type with a trivial destructor a type whose destructor is known to have no observable effects.
```

NOTE: I haven't found a way to break `is_trivially_destructible` yet, so maybe it doesn't need to change. Here I'm just making it consistent with the others.

This proposal breaks a lot of eggs, but it makes a good omelet.

It would allow the following naïve code to Just Work, even though it is buggy today ([Godbolt](#)):

```
template<class T, class U>
U *simple_copy(T *first, T *last, U *dfirst) {
    static_assert(std::is_assignable_v<T&, U&>);
    if constexpr (std::is_trivially_assignable_v<T&, U&>) {
```

```

    static_assert(sizeof(T) == sizeof(U));
    std::memmove(dfirst, first, (last - first) * sizeof(T));
    return dfirst + (last - first);
} else {
    while (first != last) *dfirst++ = *first++;
}
}

int main() {
    int source[] = {1,2,3};
    float dest[] = {1,2,3};
    simple_copy(source, source+3, dest);
    printf("%f %f %f\n", dest[0], dest[1], dest[2]);
}

```

NOTE: Okay, to really make `std::copy` work, you also need a special case for `n == 1` and/or `__datasizeof`, because a single-object range might consist of a single potentially overlapping subobject. But vendors already know that pitfall, and deal with it. See [my blog post of July 2018](#).

This proposal opens the door for vendor-specific extensions to permit e.g. `is_trivially_constructible<unique_ptr<int>, int*>`. This is perfectly safe by the library-writer's intuition, since `uninitialized_copy`'ing a range of `unique_ptr<int>` from a range of `int*` can indeed be safely lowered to `memcpy`. How should the author of `unique_ptr` warrant triviality to the compiler, so that the compiler can reflect it back to the author of `uninitialized_copy`?—We should leave that up to the vendor to figure out; but one possible extension would be

```

struct UniquePtr {
    int *p_;
    [[clang::trivial]] explicit UniquePtr(int *p);
    ~UniquePtr();
};

```

I believe this proposal completely eliminates all references to "trivial operations" and "trivial functions" (other than trivial special members). That's nice, because the Standard never defines what those are. By removing all references to them, we *never* have to define what they are.

By permitting vendors to define `is_trivially_copy_constructible<FinalPolymorphic>`, this proposal solves an issue recently raised for Clang by Haojian Wu; see [\[BitwiseCopyable\]](#). Library

authors really want to be able to memcopy and/or start-the-lifetime-of polymorphic types. That works in practice, but today it's technically UB because polymorphic types never have any trivial constructors and therefore are never implicit-lifetime types ([\[class.prop\]/9](#)). But the compiler knows that a **final** polymorphic type can be trivially copied, and therefore could consider it to be implicit-lifetime. This could solve a large swath of Haojian's problem without needing Clang to provide a separate builtin, although it probably doesn't solve the whole problem.

NOTE: Non-**final** polymorphic types can never be trivially copied nor trivially assigned, because of slicing. See [my blog post of June 2023](#).

Finally, we need to do something with [\[basic.types.general\]/2–3](#). Given the proposed wording above, I think we could just strike [\[basic.types.general\]/2–3](#) as redundant: obviously, *if* a type is trivially copy-assignable by the new wording (let alone TC), then copying its object representation also copies its value, by definition. Striking those two paragraphs would also save us from having to define what the "[original] value" of an object means, standardese-wise.

[\[basic.types.general\]/4](#) says: "For trivially copyable types, the *value representation* is a set of bits in the object representation that determines a value, which is one discrete element of an implementation-defined set of values." This sentence doesn't seem useful, but also isn't harmful; we can let it be.

§ 5.1. Conclusion

This proposal is drastic and breaks a lot of eggs, but makes a good omelet. We would come out the other side with a type-traits library that is actually usable by library-writers. Library-writers are already quietly assuming that the traits work this way, so bringing the Standard into line with those assumptions would immediately close a lot of library bugs.

I plan to bring an R1 with formal wording, and hope to also gain some implementation experience in my fork of Clang. Assistance will be gratefully accepted.

§ 6. Acknowledgments

Thanks to Ville Voutilainen and Giuseppe D'Angelo for their encouragement and discussion.

Thanks to the regulars on the cplusplus Slack for rubber-ducking a lot of these examples.

§ References

§ Informative References

[BitwiseCopyable]

Haojian Wu. *[clang] Implement a bitwise_copyable builtin type trait*. March 2024—. URL: <https://github.com/llvm/llvm-project/pull/86512>

[CWG2463]

Daveed Vandevoorde. *Trivial copyability and unions with non-trivial members*. November 2020—. URL: <https://cplusplus.github.io/CWG/issues/2463.html>

[Müller]

Jonathan Müller. *Trivially copyable does not mean trivially copy constructible*. March 2021. URL: <https://www.fooanathan.net/2021/03/trivially-copyable/>

[ODwyer]

Arthur O'Dwyer. *Types that falsely advertise trivial copyability*. May 2024. URL: <https://quux-plusone.github.io/blog/2024/05/15/false-advertising/>

[P2782]

Giuseppe D'Angelo. *A type trait to detect if value initialization can be achieved by zero-filling*. January 2023. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2782r0.html>

[P3247]

Jens Maurer. *Deprecate the notion of trivial types*. April 2024. URL: <https://open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3247r0.html>